

CONTRIBUTING

Contributing to Containers

Prerequisites

- Go 1.24 or later
- Docker and/or Podman installed (for integration tests)
- Git with SSH access configured
- kubectl (optional, for Kubernetes testing)

Getting Started

1. Clone the repository:

```
git clone <ssh-url>  
cd Containers
```

2. Verify the build:

```
go build ./...
```

3. Run all tests:

```
go test ./... -count=1 -race
```

Development Workflow

Branch Naming

Create a branch from `main` using the conventional prefix:

Prefix	Use
feat/	New features
fix/	Bug fixes
refactor/	Code restructuring without behavior change

Prefix	Use
test/	Adding or improving tests
docs/	Documentation changes
chore/	Build, CI, tooling changes

Examples:

```
git checkout -b feat/kubernetes-runtime-support
git checkout -b fix/podman-health-check-timeout
git checkout -b test/lazy-boot-integration
```

Commit Conventions

Use Conventional Commits with package scope:

<type>(<scope>): <description>

Scopes correspond to package names: runtime, compose, health, boot, lifecycle, monitor, discovery, event, or containers for cross-cutting changes.

Examples:

```
feat(runtime): add Kubernetes runtime implementation
fix(health): prevent timeout on slow health checks
test(boot): add parallel service startup test
docs(containers): update architecture diagrams
refactor(lifecycle): extract state machine to separate file
```

Code Style

Follow standard Go conventions:

- Format with gofmt (or goimports).
- Imports grouped with blank line separators: stdlib, third-party, internal.
- Line length at most 100 characters.
- Naming: camelCase for unexported, PascalCase for exported, UPPER_SNAKE_CASE for constants, acronyms all-caps (ID, HTTP, URL).
- Receivers: 1-2 letter abbreviations (bm for BootManager, rt for runtime, hc for health checker, lm for lifecycle manager).
- Always check errors, wrap with `fmt.Errorf("...: %w", err)`.
- Use defer for cleanup.
- Add doc comments to all exported types, functions, and methods.

Writing Tests

- Use table-driven tests with the testify library (assert, require).

- Naming convention: Test<Struct>_<Method>_<Scenario>.
- Test both success and failure paths.
- Use -tags=integration for tests requiring Docker/Podman.
- Test concurrent access with goroutines and race detection.
- Mock external dependencies in unit tests.

Example:

```
func TestBootManager_Start_RequiredServiceFails(t *testing.T) {
    runtime := &mockRuntime{
        startFunc: func(ctx context.Context, id string) error {
            return errors.New("start failed")
        },
    }

    manager := boot.NewBootManager(boot.WithRuntime(runtime))
    manager.AddService("postgres", boot.ServiceConfig{
        Required: true,
    })

    err := manager.Start(context.Background())
    require.Error(t, err)
    assert.Contains(t, err.Error(), "required service failed")
}
```

Running Tests

All tests with race detection

```
go test ./... -count=1 -race
```

Unit tests only (no Docker required)

```
go test ./... -short
```

Integration tests (requires Docker/Podman)

```
go test -tags=integration ./...
```

Specific package

```
go test -v ./pkg/boot/
```

Specific test

```
go test -v -run TestBootManager_Start ./pkg/boot/
```

Benchmarks

```
go test -bench=. ./tests/benchmark/
```

Coverage report

```
go test ./... -coverprofile=coverage.out
```

```
go tool cover -html=coverage.out -o coverage.html
```

Pre-Commit Checklist

Before creating a pull request, verify:

1. **Format:** `gofmt -l .` returns no files.
2. **Vet:** `go vet ./...` passes cleanly.
3. **Tests:** `go test ./... -count=1 -race` passes.
4. **Integration Tests:** `go test -tags=integration ./...` passes (if applicable).
5. **Coverage:** No reduction in test coverage.

```
gofmt -w . && go vet ./... && go test ./... -count=1 -race
```

Adding a New Runtime

1. Create runtime implementation in `pkg/runtime/<runtime_name>.go`:

```
package runtime

type LXCRuntime struct {
    client *lxc.Client
}

func NewLXCRuntime(opts ...LXCOption) (ContainerRuntime,
    error) {
    // Implementation
}

func (r *LXCRuntime) Start(ctx context.Context, containerID
    string) error {
    // Implementation
}

// Implement all ContainerRuntime interface methods
```

2. Update auto-detection in `pkg/runtime/autodetect.go`:

```
func AutoDetect(ctx context.Context) (ContainerRuntime, error)
{
    // Try Docker
    if dockerAvailable() {
        return NewDockerRuntime()
    }

    // Try Podman
    if podmanAvailable() {
        return NewPodmanRuntime()
    }

    // Try LXC
```

```

    if lxcAvailable() {
        return NewLXCRuntime()
    }

    return nil, ErrNoRuntimeAvailable
}

```

3. Add tests to pkg/runtime/<runtime_name>_test.go.

4. Run tests: `go test -v ./pkg/runtime/`

Adding a New Health Check Strategy

1. Create health check in pkg/health/<strategy>.go:

```

package health

// RedisCheck returns a health check that validates Redis
// availability
// by executing a PING command.
func RedisCheck(host string, port int, password
    string, timeout time.Duration) HealthCheck {
    return func(ctx context.Context) error {
        // Implementation using Redis client
        client := redis.NewClient(&redis.Options{
            Addr:      fmt.Sprintf("%s:%d", host, port),
            Password: password,
            Timeout:   timeout,
        })
        defer client.Close()

        return client.Ping(ctx).Err()
    }
}

```

2. Add tests to pkg/health/<strategy>_test.go:

```

func TestRedisCheck(t *testing.T) {
    // Mock Redis server or use testcontainers
    // table-driven tests ...
}

```

3. Run tests: `go test -v ./pkg/health/`

4. Update NewDefaultChecker() to include the new strategy if appropriate.

Adding Lifecycle Features

1. Add feature to pkg/lifecycle/<feature>.go:

```

package lifecycle

// AutoRestart automatically restarts services that exit
// unexpectedly
func (lm *LifecycleManager) EnableAutoRestart(maxRestarts int,
    cooldown time.Duration) {
    lm.autoRestartEnabled = true
    lm.maxRestarts = maxRestarts
    lm.restartCooldown = cooldown

    go lm.watchForUnexpectedExits()
}

```

2. Add corresponding tests.
3. Update LifecycleManager interface if adding new public methods.

Testing with Different Runtimes

Docker Testing

```

# Ensure Docker is running
docker ps

# Run integration tests
go test -tags=integration ./pkg/runtime -run TestDockerRuntime

```

Podman Testing

```

# Ensure Podman is running
podman ps

# Set environment variable to force Podman
CONTAINER_RUNTIME=podman go test -tags=integration ./pkg/runtime
    -run TestPodmanRuntime

```

Kubernetes Testing

```

# Ensure kubectl is configured
kubectl cluster-info

# Run Kubernetes tests
go test -tags=integration ./pkg/runtime -run TestKubernetesRuntime

```

Pull Request Process

1. Create a branch with the appropriate prefix.

2. Make changes, commit with conventional commit messages.
3. Ensure all tests pass (unit + integration).
4. Update documentation if adding new features.
5. Push the branch and create a pull request against main.
6. PR title should follow the same conventional commit format.
7. Describe:
 - What changed
 - Why the change was necessary
 - How to test it
 - Which runtimes are affected
8. Wait for review approval before merging.

Reporting Issues

When reporting a bug, include:

- Go version (`go version`)
- Container runtime and version (`docker version` or `podman version`)
- OS and architecture
- Minimal reproduction case (ideally with Docker Compose file)
- Expected vs. actual behavior
- Logs from the affected service
- Output from `go test -v <package>` if applicable

Performance Considerations

- Use connection pooling for runtime clients
- Implement health check caching where appropriate
- Minimize blocking operations in lifecycle hooks
- Use goroutines for parallel operations with proper synchronization
- Profile before optimizing: `go test -bench=. -cpuprofile=cpu.prof`

Release Process

1. Ensure all tests pass on all supported runtimes
2. Update CHANGELOG.md with changes
3. Tag the release: `git tag v1.x.x`
4. Push tags: `git push --tags`
5. CI will automatically build and publish

Last Updated: February 10, 2026 **Version:** 1.0.0